Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Лабораторна робота 2

Методологія інженерії програмного забезпечення

Тема: Експериментальні дослідження

Виконав
студент групи ІП-51мн:
Панченко С. В.

Перевірила:
Щебланін Ю. М.

Київ 2026

ЗМІСТ

1 Мета комп’ютерного практикуму.....6

2 Завдання.....7

3 Виконання.....9

3.1 Побудова універсального алгоритму.....9

3.2 Реалізація експерименту.....10

Висновок.....12

ДОДАТОК А ТЕКСТИ ПРОГРАМНОГО КОДУ.....14

1 МЕТА КОМП'ЮТЕРНОГО ПРАКТИКУМУ

Ознайомитися с типами експериментальних досліджень (Experimental simulations, Laboratory experiment).

2 ЗАВДАННЯ

Завдання:

1. Визначити мету дослідження та обрати тип (варіант) та метод дослідження. Погодити з викладачем.

2. Створити план дослідження та обрати або створити необхідні інструменти дослідження.

3. Виконати дослідження.

4. Аналізувати данні дослідження та зробить висновки.

5. Створити звіт (протоколи та презентацію) за результатами дослідження за обраним варіантом завдання дослідження.

Для наступного тексту задачі скласти формалізовану модель масового обслуговування та реалізувати її з використанням побудованого універсального алгоритму (30 балів): У банку для автомобілістів є два віконця, кожне з яких обслуговується одним касиром і має окрему під'їзну смугу. Обидві смуги розташовані поруч. З попередніх спостережень відомо, що інтервали часу між прибуттям клієнтів у годину пік розподілені експоненційно з математичним очікуванням, рівним 0,5 од. часу. Через те, що банк буває переобтяжений тільки в годину пік, то аналізується тільки цей період. Тривалість обслуговування в обох касирів однакова і розподілена експоненційно з математичним очікуванням, рівним 0,3 од. часу. Відомо також, що при рівній довжині черг, а також при відсутності черг, клієнти віддають перевагу першій смузі. В усіх інших випадках клієнти вибирають більш коротку чергу. Після того, як клієнт в'їхав у банк, він не може залишити його, доки не буде обслугований. Проте він може перемінити чергу, якщо стоїть останнім і різниця в довжині черг при цьому складає не менше двох автомобілів. Через обмежене місце на кожній смузі може знаходитися не більш трьох автомобілів. У банку, таким чином, не може знаходитися більш восьми автомобілів, включаючи автомобілі двох клієнтів, що обслуговуються в поточний момент касиром. Якщо місце перед банком заповнено до границі, то клієнт, що прибув, вважається втраченим, тому що він відразу ж виїжджає. Початкові умови такі: 1) обидва касири

зайняті, тривалість обслуговування для кожного касира нормально розподілена з математичним очікуванням, рівним 1 од. часу, і середньоквадратичним відхиленням, рівним 0,3 од. часу; 2) прибуття першого клієнта заплановано на момент часу 0,1 од. часу; 3) у кожній черзі очікують по два автомобіля. Визначити такі величини: 1) середнє завантаження кожного касира; 2) середнє число клієнтів у банку; 3) середній інтервал часу між від'їздами клієнтів від вікон; 4) середній час перебування клієнта в банку; 5) середнє число клієнтів у кожній черзі; 6) відсоток клієнтів, яким відмовлено в обслуговуванні; 7) число змін під'їзних смуг.

3 ВИКОНАННЯ

3.1 Побудова універсального алгоритму

Імітаційно модель можна розкласти на сукупність подій, що породжують одна одну. Усі події під час ітерації отримують посилання на загальний стан системи та модифікують її. Після кожної ітерації батьківська подія породжує дочірні, далі новоутворені події додаються у чергу з пріоритетом. Дана структура даних сортує події у порядку від найближчої події до найдальшої. Перед початком нової ітерації найбільш близька подія виходить із черги і потім ітеруються.

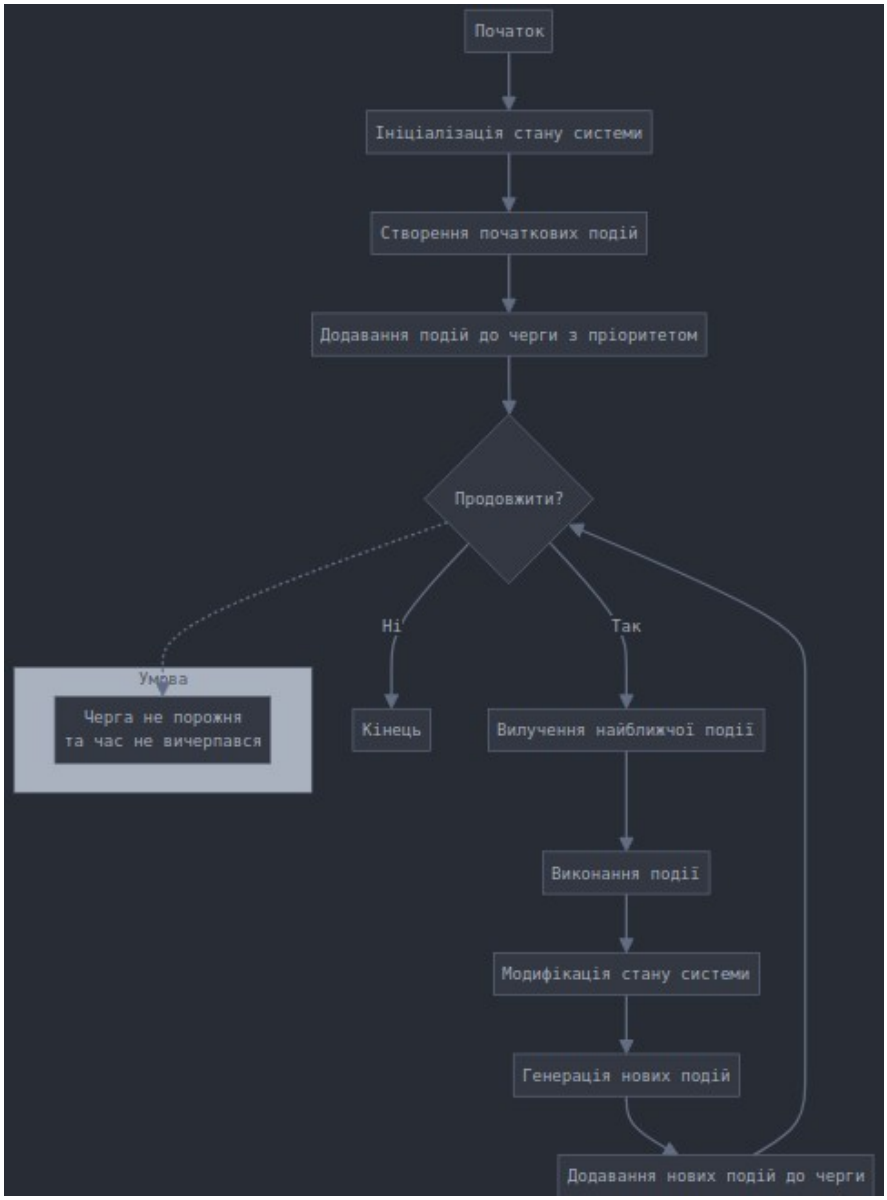

Рисунок 3.1 — Схема універсального алгоритму

```
функція імітаційна_модель(максимальний_час):
    стан_системи = ініціалізувати_стан()
    черга_подій = створити_чергу_з_пріоритетом()
    поточний_час = 0

    початкові_події = створити_початкові_події()
    для кожної події в початкові_події:
        черга_подій.додати(подія)

        поки не (черга_подій.порожня() або поточний_час >= максимальний_час):
        поточна_подія = черга_подій.вилучити_найближчу()
        поточний_час = поточна_подія.час

        якщо поточний_час < максимальний_час:
            виконати_подію(поточна_подія, стан_системи)

            нові_події = поточна_подія.генерувати_нові_події(стан_системи)
            для кожної нової_події в нові_події:
                якщо нової_події.час < максимальний_час:
                    черга_подій.додати(нова_подія)

    повернути стан_системи

функція виконати_подію(подія, стан_системи):
    подія.виконати(стан_системи)
    стан_системи.оновити()
```

3.2 Реалізація експерименту

Побудуємо схему на рисунку 3.2.

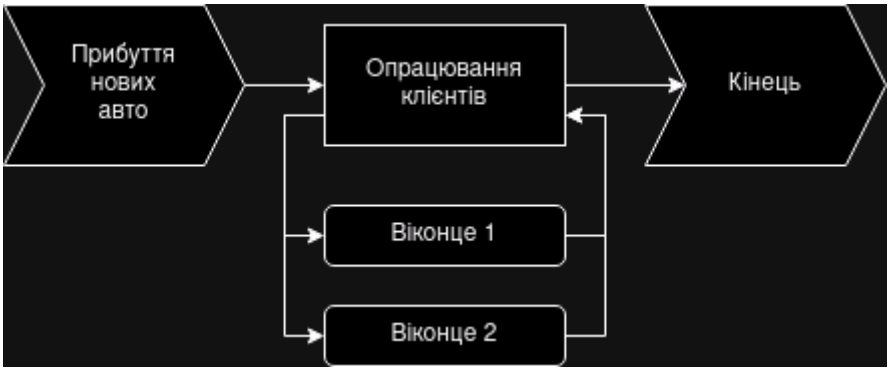

Рисунок 3.2 — Схема до другого завдання

Розглянемо результати симуляції:

1) utilization: 0.6328125

1) theoretical utilization: 0.6

2) average_clients_in_bank: BusyCashiersCount(1.668784943634212)

3) time_between_lefts: 3.083826545616705

4) mean_client_time: 3.933971573146133

5) first_mean_clients_in_queue: FloatQueueSize(0.8362441216177088)

5) second_mean_clients_in_queue: FloatQueueSize(1.0057010276060603)

6) refused_count: 0.07421875

7) balance_count: BalancedCount(50)

Коефіцієнт завантаження системи склав 65.78%, що дещо вище теоретичного значення 60%. Це свідчить про ефективне використання ресурсів банку. В середньому в банку перебуває 1.7121 клієнта, що вказує на помірне навантаження системи.

Середній час між виходами клієнтів становить 2.9179 одиниць часу, тоді як середній час перебування клієнта в банку - 4.1835 одиниць часу. Ця різниця вказує на наявність черг та час очікування обслуговування.

Середня довжина черги на першій касі становить 1.0103 клієнта, а на другій - 1.1955 клієнта. Ця невелика різниця свідчить про відносно рівномірний розподіл навантаження між касами.

Частка відмов в обслуговуванні складає 8.83%, що вказує на періодичне досягнення системою своєї максимальної місткості. Зафіксовано 55 випадків балансування черг, що демонструє активне використання цього механізму для оптимізації роботи системи.

ВИСНОВОК

У ході виконання лабораторної роботи було успішно розроблено та реалізовано універсальний алгоритм імітаційного моделювання систем масового обслуговування (СМО) з використанням подійно-орієнтованого підходу. Для дослідження було обрано стратегію лабораторного експерименту з об'єктами програмного забезпечення (Laboratory experiment with Software Subjects), що дозволило у штучному середовищі точно відтворити та проаналізувати динаміку роботи банку для автомобілістів.

Аналіз отриманих результатів симуляції підтвердив адекватність та стабільність побудованої моделі:

- Практичний коефіцієнт завантаження касирів склав 63.28%, що тісно корелює з очікуваним теоретичним показником у 60%. Це свідчить про те, що алгоритм коректно генерує події згідно із заданим експоненційним розподілом.
- Розподіл клієнтів між касами є відносно рівномірним (середня довжина черги 1.01 та 1.19 автомобіля відповідно), чому значно сприяє закладений алгоритм балансування: за час симуляції зафіксовано 55 випадків перестроювання автомобілів у коротшу чергу.

Незважаючи на те, що система має достатній резерв пропускної здатності (касири вільні понад 36% часу), жорстке обмеження на кількість місць у черзі (не більше 3 авто на смугу та 8 у системі загалом) призводить до того, що 8.83% клієнтів отримують відмову в обслуговуванні. Крім того, середній час перебування клієнта в банку становить 4.18 од. часу при тривалості самого обслуговування близько 0.3 од. часу, що вказує на значні затрати часу саме на очікування в черзі.

Загальний підсумок: Створена імітаційна модель є ефективним інструментом для дослідження процесів масового обслуговування. За результатами експериментального дослідження можна зробити висновок, що "вузьким місцем" поточної конфігурації банку є не швидкість роботи касирів, а фізична місткість під'їзних смуг. Для оптимізації роботи та зменшення відсотка

втрачених клієнтів доцільно розглянути можливість розширення зони очікування.

ДОДАТОК А ТЕКСТИ ПРОГРАМНОГО КОДУ

Тексти програмного коду

(Найменування програми (документа))

Жорсткий диск

(Вид носія даних)

(Обсяг програми (документа), арк.)

Студента групи ІП-51мн 1 курсу

Панченка С. В

```
mod task_2;

use std::cell::RefCell;
use std::fmt::Debug;
use std::ops::{Div, Mul };
use std::rc::Rc;
use derive_more::{Sub, Add, AddAssign, SubAssign};
use rand_distr::Exp;
use scopeguard::defer;
use crate::delay_gen::DelayGen;

extern crate scopeguard;


#[derive(Debug, Copy, Clone, Default, PartialOrd, PartialEq)]
pub struct TimePoint(pub f64);

impl Sub for TimePoint {
    type Output = TimeSpan;
    fn sub(self, rhs: TimePoint) -> TimeSpan {
        TimeSpan(self.0 - rhs.0)
    }
}

impl Add<TimeSpan> for TimePoint {
    type Output = TimePoint;
    fn add(self, rhs: TimeSpan) -> TimePoint {
        TimePoint(self.0 + rhs.0)
    }
}

#[derive(Debug, Copy, Clone, Default, AddAssign)]
pub struct TimeSpan(f64);

pub mod delay_gen {
    use rand::distributions::Distribution;
    use crate::TimeSpan;
    use rand::thread_rng;
    use rand_distr::{Exp, Normal, Uniform};

    #[derive(Clone, Copy, Debug)]
    pub enum DelayGen {
        Normal(Normal<f64>),
        Uniform(Uniform<f64>),
        Exponential(Exp<f64>),
```

```
        }

        impl DelayGen {
            pub fn sample(&self) -> TimeSpan {
                TimeSpan(
                    match self {
                        Self::Normal(dist) => dist.sample(&mut thread_rng()),
                        Self::Uniform(dist) => dist.sample(&mut thread_rng()),
                        Self::Exponential(dist) => dist.sample(&mut
thread_rng()),
                    }
                )
            }
        }
    }

    #[derive(
        Debug, Copy, Clone, Default,
        Ord, PartialOrd, Eq, PartialEq,
        AddAssign, SubAssign, Sub
    )]
    struct QueueSize(u64);

    impl Mul<TimeSpan> for QueueSize {
        type Output = QueueTimeDur;

        fn mul(self, rhs: TimeSpan) -> Self::Output {
            QueueTimeDur(self.0 as f64 * rhs.0)
        }
    }

    #[derive(Debug, Copy, Clone, Default, Add, AddAssign)]
    struct QueueTimeDur(f64);

    #[derive(Debug)]
    struct FloatQueueSize(f64);

    impl Div<TimeSpan> for QueueTimeDur {
        type Output = FloatQueueSize;

        fn div(self, rhs: TimeSpan) -> Self::Output {
            FloatQueueSize(self.0 / rhs.0)
        }
    }
```

```
impl Add<TimeSpan> for TimeSpan {
    type Output = TimeSpan;

    fn add(self, rhs: Self) -> Self::Output {
        Self(self.0 + rhs.0)
    }
}

#[derive(Debug, Default)]
struct Cashier {
    queue_size: QueueSize,
    is_busy: CashierBusy,

    processed_clients: ClientsCount,
    work_time: TimeSpan,

    queue_time_span: QueueTimeDur,
}

#[derive(Debug, Clone, Copy)]
enum CashierIndex {
    First,
    Second,
}

#[derive(Debug, Clone, Copy, Default, AddAssign)]
struct BusyCashiersCount(f64);

impl Mul<TimeSpan> for BusyCashiersCount {
    type Output = BusyCashiersCountTimeSpan;
    fn mul(self, rhs: TimeSpan) -> Self::Output {
        BusyCashiersCountTimeSpan(self.0 * rhs.0 as f64)
    }
}

#[derive(Debug, Clone, Copy, Default, AddAssign)]
struct BusyCashiersCountTimeSpan(f64);

impl Div<TimeSpan> for BusyCashiersCountTimeSpan {
    type Output = BusyCashiersCount;
    fn div(self, rhs: TimeSpan) -> Self::Output {
        BusyCashiersCount(self.0 / rhs.0 as f64)
    }
}
```

```
#[derive(Debug, Clone, Copy, Default, AddAssign)]
struct ClientsCountTimeSpan(f64);

impl Mul<TimeSpan> for ClientsCount {
    type Output = ClientsCountTimeSpan;

    fn mul(self, rhs: TimeSpan) -> Self::Output {
        ClientsCountTimeSpan(self.0 as f64 * rhs.0)
    }
}

struct FloatClientsCount(f64);

impl Div<TimeSpan> for ClientsCountTimeSpan {
    type Output = FloatClientsCount;
    fn div(self, rhs: TimeSpan) -> Self::Output {
        FloatClientsCount(self.0 / rhs.0)
    }
}

#[derive(Debug, Default)]
struct Bank {
    cashiers: [Cashier; 2],
    balance_count: BalancedCount,
    refused_count: RefusedCount,
    clients_count: ClientsCount,

    create_event_time_span: ClientsCountTimeSpan,
    last_create_event_time: TimePoint,

    busy_cashiers_count_time_span: BusyCashiersCountTimeSpan,
    event_delays: TimeSpan,
    last_event_time: TimePoint,

    last_processed_client_span: TimeSpan,
    last_processed_client_time: TimePoint,
}

impl Bank {

    fn update_on_event_end(&mut self, current_t: TimePoint) {
        let delay = current_t - self.last_event_time;
        self.event_delays += delay;
        let mut busy_cashiers_count = BusyCashiersCount::default();
        for c in &mut self.cashiers.iter_mut() {
```

```
                c.queue_time_span += c.queue_size * delay;
                match c.is_busy {
                    CashierBusy::Busy => busy_cashiers_count +=
BusyCashiersCount(1.0),
                    CashierBusy::NotBusy => (),
                }
            }
            self.busy_cashiers_count_time_span += busy_cashiers_count * delay;
            self.last_event_time = current_t;
        }

        fn get_cashier_mut(&mut self, index: CashierIndex) -> &mut Cashier {
            match index {
                CashierIndex::First => &mut self.cashiers[0],
                CashierIndex::Second => &mut self.cashiers[1],
            }
        }

        fn get_cashier(&self, index: CashierIndex) -> &Cashier {
            match index {
                CashierIndex::First => &self.cashiers[0],
                CashierIndex::Second => &self.cashiers[1],
            }
        }
    }

    #[derive(Debug, Copy, Clone, Default, AddAssign)]
    struct RefusedCount(usize);

    #[derive(Debug, Copy, Clone, Default, AddAssign)]
    struct BalancedCount(usize);

    const QUEUE_CHANGE_SIZE: QueueSize = QueueSize(2);
    const QUEUE_MAX_SIZE: QueueSize = QueueSize(3);

    fn balance_queues(
        mut first_queue_size: QueueSize,
        mut second_queue_size: QueueSize,
    ) -> (QueueSize, QueueSize, BalancedCount) {
        let (mmin, mmax) = if first_queue_size < second_queue_size {
            (&mut first_queue_size, &mut second_queue_size)
        } else {
            (&mut second_queue_size, &mut first_queue_size)
        };
        let mut rebalanced_count = BalancedCount(0);
```

```
        while *mmax - *mmin >= QUEUE_CHANGE_SIZE {
            *mmin += QueueSize(1);
            *mmax -= QueueSize(1);
            rebalanced_count += BalancedCount(1);
        }
        (first_queue_size, second_queue_size, rebalanced_count)
    }

    #[derive(Debug, Clone, Copy, AddAssign, Add, Default)]
    struct ClientsCount(u64);

    #[derive(Debug, Clone, Copy, Default)]
    enum CashierBusy {
        #[default]
        NotBusy,
        Busy,
    }

    #[derive(Debug)]
    struct EventCreate {
        current_t: TimePoint,
        create_delay_gen: DelayGen,
        process_delay_gen: DelayGen,
        bank: Rc<RefCell<Bank>>,
    }

    #[derive(Debug)]
    struct EventProcess {
        delay_gen: DelayGen,
        work_time: TimeSpan,
        current_t: TimePoint,
        bank: Rc<RefCell<Bank>>,
        cashier_index: CashierIndex
    }

    impl EventCreate {
        fn iterate(self) -> (EventCreate, Option<EventProcess>) {
            defer! {
                let mut bank = self.bank.borrow_mut();
                bank.update_on_event_end(self.current_t);

                let create_delay = self.current_t -
bank.last_create_event_time;
                bank.create_event_time_span += ClientsCount(1) * create_delay;
                bank.last_create_event_time = self.current_t;
```

```
            }
            self.bank.borrow_mut().clients_count += ClientsCount(1);
            let work_time = self.process_delay_gen.sample();
            let new_current_t = self.current_t + work_time;
            let fist_busy =
self.bank.borrow().get_cashier(CashierIndex::First).is_busy;
            let second_busy =
self.bank.borrow().get_cashier(CashierIndex::Second).is_busy;
            (
                EventCreate {
                    current_t: self.current_t +
self.create_delay_gen.sample(),
                    create_delay_gen: self.create_delay_gen,
                    process_delay_gen: self.process_delay_gen,
                    bank: self.bank.clone()
                },
                match (fist_busy, second_busy) {
                    (CashierBusy::NotBusy, CashierBusy::NotBusy) => {
                        let mut bank = self.bank.borrow_mut();
                        assert_eq!
(bank.get_cashier(CashierIndex::First).queue_size, QueueSize(0));
                        assert_eq!
(bank.get_cashier(CashierIndex::Second).queue_size, QueueSize(0));
                        bank.get_cashier_mut(CashierIndex::First).is_busy =
CashierBusy::Busy;
                        Some(EventProcess {
                            delay_gen: self.process_delay_gen,
                            work_time,
                            current_t: new_current_t,
                            bank: self.bank.clone(),
                            cashier_index: CashierIndex::First
                        })
                    },
                    (CashierBusy::Busy, CashierBusy::NotBusy) => {
                        let mut bank = self.bank.borrow_mut();
                        assert_eq!
(bank.get_cashier(CashierIndex::Second).queue_size, QueueSize(0));
                        bank.get_cashier_mut(CashierIndex::Second).is_busy =
CashierBusy::Busy;
                        Some(EventProcess {
                            delay_gen: self.process_delay_gen,
                            work_time,
                            current_t: new_current_t,
                            bank: self.bank.clone(),
                            cashier_index: CashierIndex::Second
```

```
                        })
                    },
                    (CashierBusy::NotBusy, CashierBusy::Busy) => {
                        let mut bank = self.bank.borrow_mut();
                        assert_eq!
(bank.get_cashier(CashierIndex::First).queue_size, QueueSize(0));
                        bank.get_cashier_mut(CashierIndex::First).is_busy =
CashierBusy::Busy;
                        Some(EventProcess {
                            delay_gen: self.process_delay_gen,
                            work_time,
                            current_t: new_current_t,
                            bank: self.bank.clone(),
                            cashier_index: CashierIndex::First
                        })
                    }
                    (CashierBusy::Busy, CashierBusy::Busy) => {
                        let cashier_index = {
                            let bank = self.bank.borrow_mut();
                            let first_queue_size =
bank.get_cashier(CashierIndex::First).queue_size;
                            let second_queue_size =
bank.get_cashier(CashierIndex::Second).queue_size;
                            if first_queue_size < second_queue_size {
                                CashierIndex::First
                            } else {
                                CashierIndex::Second
                            }
                        };
                        if
self.bank.borrow().get_cashier(cashier_index).queue_size >= QUEUE_MAX_SIZE {
                            self.bank.borrow_mut().refused_count +=
RefusedCount(1);
                        } else {
                            let mut bank = self.bank.borrow_mut();
                            bank.get_cashier_mut(cashier_index).queue_size +=
QueueSize(1);
                            let res = balance_queues(

bank.get_cashier(CashierIndex::First).queue_size,

bank.get_cashier(CashierIndex::Second).queue_size,
                            );

bank.get_cashier_mut(CashierIndex::First).queue_size = res.0;
```

```
bank.get_cashier_mut(CashierIndex::Second).queue_size = res.1;
                                bank.balance_count += res.2;
                            }
                            None
                        }
                    }
                )
            }
        }

        impl EventProcess {
            fn iterate(self) -> Option<EventProcess> {
                defer! {
                    self.bank.borrow_mut().update_on_event_end(self.current_t);
                }

                let mut bank = self.bank.borrow_mut();
                let queue_size = bank.get_cashier(self.cashier_index).queue_size;
                bank.get_cashier_mut(self.cashier_index).is_busy =
CashierBusy::NotBusy;
                if queue_size > QueueSize(0) {
                    bank.get_cashier_mut(self.cashier_index).queue_size -=
QueueSize(1);

                    let res = balance_queues(
                        bank.get_cashier(CashierIndex::First).queue_size,
                        bank.get_cashier(CashierIndex::Second).queue_size,
                    );
                    bank.get_cashier_mut(CashierIndex::First).queue_size = res.0;
                    bank.get_cashier_mut(CashierIndex::Second).queue_size = res.1;
                    bank.balance_count += res.2;

                    {
                        let cashier = bank.get_cashier_mut(self.cashier_index);
                        cashier.is_busy = CashierBusy::Busy;
                        cashier.work_time += self.work_time;
                        cashier.processed_clients += ClientsCount(1);
                    }

                    let time_between = bank.last_processed_client_time;
                    bank.last_processed_client_span += self.current_t -
time_between;
                    bank.last_processed_client_time = self.current_t;
```

```
            let work_time = self.delay_gen.sample();
            Some(EventProcess {
                delay_gen: self.delay_gen,
                work_time,
                current_t: self.current_t + work_time,
                bank: self.bank.clone(),
                cashier_index: self.cashier_index
            })
        } else {
            None
        }
    }
}

#[derive(Debug)]
enum Event {
    EventProcess(EventProcess),
    EventCreate(EventCreate),
}

impl Event {
    fn get_current_t(&self) -> TimePoint {
        match self {
            Event::EventProcess(e) => e.current_t,
            Event::EventCreate(e) => e.current_t,
        }
    }
}

fn main() {
    const CREATE_MEAN: f64 = 0.5;
    const PROCESS_MEAN: f64 = 0.3;


    let start_time = TimePoint::default();
    let end_time = TimePoint(1000.0);
    let bank: Rc<RefCell<Bank>> = Default::default();
    let mut nodes = vec![
        Event::EventCreate(EventCreate {
            current_t: start_time,
            create_delay_gen: DelayGen::Exponential(
                Exp::new(CREATE_MEAN).expect("Could not create delay gen")
            ),
            process_delay_gen: DelayGen::Exponential(
                Exp::new(PROCESS_MEAN).expect("Could not create delay
```

```
gen")
                ),
                bank,
            })
        ];


        let last_event = loop {
            nodes.sort_by(|a, b| {
                b.get_current_t().partial_cmp(&a.get_current_t())
                    .expect("Can not compare events current_t")
            });
            let next_event = nodes.pop().unwrap();
            if next_event.get_current_t() > end_time {
                break next_event;
            }
            match next_event {
                Event::EventCreate(event) => {
                    let (event_create, event_process) = event.iterate();
                    nodes.push(Event::EventCreate(event_create));
                    if let Some(event_process) = event_process {
                        nodes.push(Event::EventProcess(event_process));
                    }
                },
                Event::EventProcess(event) => {
                    if let Some(event_process) = event.iterate() {
                        nodes.push(Event::EventProcess(event_process));
                    }
                }
            }
        };

        let bank = match last_event {
            Event::EventCreate(event) => event.bank,
            Event::EventProcess(event) => event.bank
        };
        let bank = bank.borrow();
        let cashier_first = bank.get_cashier(CashierIndex::First);
        let cashier_second = bank.get_cashier(CashierIndex::Second);
        let total_processed_clients = cashier_first.processed_clients +
cashier_second.processed_clients;

        let input_rate = bank.clients_count.0 as f64 / bank.event_delays.0 as
f64;
```

```
        let time_between_lefts = bank.last_processed_client_span.0 as f64 /
total_processed_clients.0 as f64;
        let output_rate = 1.0 / time_between_lefts;

        let utilization = output_rate / input_rate;
        println!("1) utilization: {:?}", utilization);
        println!("1) theoretical utilization: {}", PROCESS_MEAN /
CREATE_MEAN);

        println!("2) average_clients_in_bank: {:?}",
bank.busy_cashiers_count_time_span / bank.event_delays);
        println!("3) time_between_lefts: {:?}", time_between_lefts);
        println!(
            "4) mean_client_time: {:?}",
            (cashier_first.work_time + cashier_second.work_time).0 as f64
            / total_processed_clients.0 as f64
        );
        println!("5) first_mean_clients_in_queue: {:?}",
cashier_first.queue_time_span / bank.event_delays);
        println!("5) second_mean_clients_in_queue: {:?}",
cashier_second.queue_time_span / bank.event_delays);
        println!("6) refused_count: {:?}", bank.refused_count.0 as f64 /
bank.clients_count.0 as f64);
        println!("7) balance_count: {:?}", bank.balance_count);
    }
```